
Algorithm 1 ECHO-Net: Environmental Sound Classification via Dual-Branch Fusion

Require: Audio Dataset $\mathcal{D}$, Initialized Local Branch (CNN14) B_L , Initialized Global Branch (HTS-AT) B_G .

Require: Learnable Choquet capacities $\gamma_L, \gamma_G, \gamma_{joint}$, class-wise temperatures T_L, T_G .

Require: Residual MLP $\mathcal{R}$ with weights $\mathbf{W}_1, \mathbf{b}_1$, zero-initialized weights $\mathbf{W}_{out} = \mathbf{0}, \mathbf{b}_{out} = \mathbf{0}$, and zero-initialized scalar $\alpha_{res} = 0$.

```
1: // Phase 1: Local Branch (PANNS CNN14 + CBAM) Training
2: for each audio sample  $x \in \mathcal{D}$  and its target  $y$  do
3:   Front-end Extraction:
4:     Extract main log-mel spectrogram (1024 window):  $x_{mel} \leftarrow \text{StandardFrontend}(x)$ 
5:     Extract auxiliary temporal spectrogram (512 window):  $x_{aux} \leftarrow \text{AuxFrontend}(x)$ 
6:   Main Backbone & CBAM Processing:
7:     Pass  $x_{mel}$  through 6 ConvBlocks to obtain hierarchical feature maps.
8:     For feature maps  $\mathbf{F}$  from blocks 4, 5, and 6:
9:       Channel Attention:  $\mathbf{M}_c(\mathbf{F}) = \sigma(\text{MLP}(\text{AvgPool}(\mathbf{F})) + \text{MLP}(\text{MaxPool}(\mathbf{F})))$ 
10:      Refine features:  $\mathbf{F}' = \mathbf{M}_c(\mathbf{F}) \otimes \mathbf{F}$ 
11:      Spatial Attention:  $\mathbf{M}_s(\mathbf{F}') = \sigma(\text{Conv}_{2D}([\text{AvgPool}(\mathbf{F}'); \text{MaxPool}(\mathbf{F}')]))$ 
12:      Final CBAM Output:  $\mathbf{F}'' = \mathbf{M}_s(\mathbf{F}') \otimes \mathbf{F}'$ 
13:      Apply Dual Pooling (Mean + Max) on  $\mathbf{F}''$  and concatenate to form main embedding  $e_{main}$ .
14:      Pass  $e_{main}$  through Fully Connected layer to get  $\mathbf{h}_{main}$ .
15:   Temporal Auxiliary Stream:
16:     Process  $x_{aux}$  via TemporalAuxEncoder (using  $7 \times 1$  deformed kernels) to get  $\mathbf{f}_{aux}$ .
17:   Zero-Initialized Fusion:
18:     Project aux features:  $\mathbf{h}_{aux} = \mathbf{W}_{aux}\mathbf{f}_{aux} + \mathbf{b}_{aux}$  (where  $\mathbf{W}_{aux} = \mathbf{0}, \mathbf{b}_{aux} = \mathbf{0}$ )
19:     Combine and classify:  $\mathbf{z}_L = \text{Classifier}(\text{ReLU}(\mathbf{h}_{main} + \mathbf{h}_{aux}))$ 
20:     Optimize  $B_L$  using Label-Smoothed Cross-Entropy Loss based on  $\mathbf{z}_L$  and target  $y$ .
21: end for
22: Extract out-of-fold validation logits  $Z_L$  for the entire dataset from the trained  $B_L$ .
23: // Phase 2: Global Branch (HTS-AT) Training with SFPL
24: for each audio sample  $x \in \mathcal{D}$  and its target  $y$  do
25:   Front-end Extraction:
26:     Extract main log-mel spectrogram (1024 window):  $x_{mel} \leftarrow \text{StandardFrontend}(x)$ 
27:     Extract auxiliary spectral spectrogram (2048 window):  $x_{aux\_spec} \leftarrow \text{AuxFrontend}(x)$ 
28:   Main Backbone & Auxiliary Stream:
29:     Pass  $x_{mel}$  through Swin Transformer to obtain global features  $\mathbf{f}_G$ .
30:     Apply Attentive TSCAM pooling:  $\mathbf{z}_{main} = \text{TSCAM}(\mathbf{f}_G)$ 
31:     Process  $x_{aux\_spec}$  via SpectralAuxEncoder (using  $1 \times 7$  deformed kernels) to get  $\mathbf{f}_{aux\_spec}$ .
32:   Zero-Initialized Logit Fusion:
33:     Project aux features directly to logits:  $\mathbf{z}_{aux} = \mathbf{W}_{spec}\mathbf{f}_{aux\_spec} + \mathbf{b}_{spec}$  (zero-initialized)
34:     Final global logits:  $\mathbf{z}_G = \mathbf{z}_{main} + \mathbf{z}_{aux}$ 
35:   Same Failure Penalty Loss (SFPL) Calculation:
36:     Retrieve local branch predictions:  $\hat{y}_L \leftarrow \arg \max(Z_L(x))$ 
37:     Current global predictions:  $\hat{y}_G \leftarrow \arg \max(\mathbf{z}_G)$ , and probabilities  $\mathbf{p}_G = \text{Softmax}(\mathbf{z}_G)$ 
38:     Compute Mutual Failure Mask:  $\mathcal{M}_{fail} = \mathcal{I}(\hat{y}_L \neq y \wedge \hat{y}_G \neq y \wedge \hat{y}_L = \hat{y}_G)$ 
39:     Compute Penalty:  $\mathcal{L}_{SFPL} = \mathcal{M}_{fail} \cdot \mathbf{p}_G[\hat{y}_L]$ 
40:     Total Loss:  $\mathcal{L}_{Total} = \mathcal{L}_{LabelSmoothedCE} + \text{Softplus}(w_{SFP}) \cdot \mathcal{L}_{SFPL}$ 
41:     Optimize  $B_G$  using  $\mathcal{L}_{Total}$ .
42: end for
43: Extract out-of-fold validation logits  $Z_G$  for the entire dataset from the trained  $B_G$ .
```

Algorithm 2 ECHO-Net: Phase 3 (Fusion)

```
1: // Phase 3: Entropy-Aware Choquet Meta-Learner
2: Freeze branches  $B_L$  and  $B_G$ .
3: for each pair of out-of-fold logits  $(z_L, z_G) \in (Z_L, Z_G)$  do
4:   Temperature Scaling & Confidence Gating:
5:   Soften logits:  $p_i = \text{Softmax}\left(\frac{z_i}{\text{Softplus}(T_i) + \epsilon}\right)$  for  $i \in \{L, G\}$ 
6:   Shannon Entropy Confidence:  $C_i = 1 - \frac{-\sum_{k=1}^K p_i[k] \log(p_i[k] + \epsilon)}{\log(K)}$ 
7:   Fuzzy Capacities (Measures):
8:   Gated capacities:  $g_L = C_L \cdot \sigma(\gamma_L)$ ,  $g_G = C_G \cdot \sigma(\gamma_G)$ 
9:   Joint capacity:  $g_{joint} = \max(g_L, g_G) + \sigma(\gamma_{joint}) \odot (1 - \max(g_L, g_G))$ 
10:  Vectorized Choquet Integral Computation (Element-wise over classes  $k$ ):
11:  if  $p_L[k] \geq p_G[k]$  then
12:     $p_{choquet}[k] = (p_L[k] - p_G[k]) \cdot g_L[k] + p_G[k] \cdot g_{joint}[k]$ 
13:  else
14:     $p_{choquet}[k] = (p_G[k] - p_L[k]) \cdot g_G[k] + p_L[k] \cdot g_{joint}[k]$ 
15:  end if
16:  Normalize  $p_{choquet}$  such that it sums to 1.
17:  Residual Correction MLP:
18:  Concatenate raw logits:  $z_{concat} = [z_L || z_G]$ 
19:  Hidden Layer:  $h_{res} = \text{Dropout}(\text{SiLU}(W_1 z_{concat} + b_1))$ 
20:  Zero-Initialized Output:  $z_{res} = W_{out} h_{res} + b_{out}$ 
21:  Correction probability vector:  $\Delta p = \text{Softmax}(z_{res})$ 
22:  Final Bounded Fusion:
23:   $p_{fused} = p_{choquet} + \sigma(\alpha_{res}) \cdot \Delta p$ 
24:  Normalize  $p_{fused}$  to ensure a valid probability distribution.
25:  Optimize Choquet parameters  $(\gamma, T, W_1, b_1, W_{out}, b_{out}, \alpha_{res})$  via Negative Log-Likelihood Loss.
26: end for
27: return Final fused probabilities  $p_{fused}$ 
```
